📂 CATEGORY 4: Text Processing

Linux & Bash Mastery — DevOps/SRE/Platform Engineer Interview Guide

---

Table of Contents

- 4.1 `grep` — Search Tool
- 4.2 `cut`, `sort`, `uniq`, `wc` — Pipeline Building Blocks
- 4.3 `tr` — Character Translation
- 4.4 `sed` — Stream Editor
- 4.5 `awk` — Field Processor & Mini Language
- 4.6 `grep` vs `sed` vs `awk` — When to Use Which
- 4.7 Advanced `awk` — Multi-rule, Associative Arrays, Formatted Output
- 4.8 Log Parsing Pipelines — Real Incident Analysis
- 4.9 `jq` — JSON Parsing for Cloud APIs
- 4.10 `perl` One-liners — When awk/sed Aren't Enough
- Master Gotcha List
- Top Interview Questions

4.1 `grep` — Search Tool

🔷 What it is in simple terms

`grep` (**G**lobal **R**egular **E**xpression **P**rint) scans input line by line and prints lines that match a pattern. Think of it as Ctrl+F for the terminal — but infinitely more powerful.

🔷 Why it exists / What problem it solves

Before `grep`, finding text in files meant writing custom programs. `grep` was created in 1973 by Ken Thompson (Bell Labs) specifically to search for regex patterns in files — making text search a first-class Unix citizen. In production, you use it constantly: finding errors in logs, searching configs, filtering command output.

🔷 How it works internally

```
Input (file or stdin)
        │
        ▼
┌──────────────────┐
│  Read line by line │
└──────────────────┘
        │
        ▼
┌──────────────────
```

```
│  Apply regex match │  ◄── Pattern compiled once, applied per line
└──────────────────┘
          │
          │
     Match? ─── YES ──► Print line to stdout
          │
         NO
          │
     Next line
```

Internally, `grep` uses one of three regex engines depending on flags:

- **BRE** (Basic Regex) — default
- **ERE** (Extended Regex) — with `-E` or `egrep`
- **PCRE** (Perl-Compatible) — with `-P`

🔷 Key flags with real examples

```
# Setup: create a sample log file
cat > /tmp/app.log << 'EOF'
2024-01-15 ERROR: disk full on /dev/sda1
2024-01-15 INFO: service started
2024-01-15 WARNING: high memory usage
2024-01-15 error: connection timeout
2024-01-15 ERROR: null pointer exception
2024-01-16 INFO: backup completed
EOF
```

Basic match

```
grep "ERROR" /tmp/app.log
# 2024-01-15 ERROR: disk full on /dev/sda1
# 2024-01-15 ERROR: null pointer exception
```

`-i` — case insensitive

```
grep -i "error" /tmp/app.log
# Matches ERROR, error, Error, eRrOr
```

`-n` — show line numbers

```
grep -n "ERROR" /tmp/app.log
# 1:2024-01-15 ERROR: disk full on /dev/sda1
```

```
# 5:2024-01-15 ERROR: null pointer exception
```

`-v` — invert match (exclude lines)

```
grep -v "INFO" /tmp/app.log
# Shows everything EXCEPT INFO lines
```

`-r` — recursive directory search

```
grep -r "ERROR" /var/log/
grep -r "DB_PASSWORD" /etc/          # Find hardcoded secrets in configs
grep -rn "TODO" /opt/app/src/        # Find all TODOs in codebase
```

`-l` — only print filenames

```
grep -rl "ERROR" /var/log/
# /var/log/app.log
# /var/log/nginx/error.log
```

`-c` — count matching lines

```
grep -c "ERROR" /tmp/app.log
# 2
```

⚠ `-A`, `-B`, `-C` — context lines (HIGH VALUE IN INTERVIEWS)

```
grep -A 3 "ERROR" /tmp/app.log   # 3 lines AFTER match
grep -B 2 "ERROR" /tmp/app.log   # 2 lines BEFORE match
grep -C 2 "ERROR" /tmp/app.log   # 2 lines BEFORE AND AFTER
# Critical for incident debugging — see what happened before/after an
error
```

`-E` — Extended regex (ERE)

```
grep -E "ERROR|WARNING" /tmp/app.log        # OR match
grep -E "^2024-01-15" /tmp/app.log           # Lines starting with date
grep -E "ERROR.+(disk|memory)" /tmp/app.log  # ERROR followed by disk or
memory
```

`-P` — Perl regex (PCRE)

```
grep -P "\d{4}-\d{2}-\d{2}" /tmp/app.log    # Match date pattern
grep -P "(?<=ERROR: ).+" /tmp/app.log         # Lookbehind
```

`-o` — print only matching part

```
grep -oE "[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" /var/log/nginx/access.log
# Extracts only IP addresses from the log
```

`-w` — whole word match

```
grep -w "error" file.txt    # Won't match "errors" or "terror"
```

Piping and chaining

```
# Count ERRORs per day
grep "ERROR" /tmp/app.log | cut -d' ' -f1 | sort | uniq -c

# Find processes listening on port 80
ss -tlnp | grep ":80"

# Search compressed logs without decompressing
zgrep "ERROR" /var/log/app.log.gz
```

🔷 Short crisp interview answer

> "`grep` searches input line-by-line for a regex pattern and prints matching lines. I use it daily for log analysis — filtering errors, extracting IPs, finding config values. Key flags I reach for are `-i` for case-insensitive, `-r` for recursive, `-n` for line numbers, `-v` to exclude patterns, `-C` for context lines, and `-E` for extended regex with OR patterns."

🔷 Deep version

> "Internally `grep` compiles the regex once and runs it against each line using one of three engines: BRE (default), ERE (`-E`), or PCRE (`-P`). The key performance implication is that grep is line-buffered and extremely fast because it uses `mmap` or read-ahead on files. For very large logs, `grep -F` (fixed string, no regex) is significantly faster since it uses Boyer-Moore string search instead of regex. In

> pipelines, `grep` is often the first filter to reduce data volume before slower tools like `awk` or `sed` process it."

🔷 Real-world production example

```
# Incident: Users reporting 500 errors. You're on-call at 2 AM.

# Step 1: How many 500s in the last hour?
grep "HTTP/1.1\" 5" /var/log/nginx/access.log | \
  awk '{print $4}' | cut -c2-14 | sort | uniq -c

# Step 2: Which endpoints are failing?
grep "\" 500 " /var/log/nginx/access.log | \
  awk '{print $7}' | sort | uniq -c | sort -rn | head -10

# Step 3: What IPs are hitting the 500s?
grep "\" 500 " /var/log/nginx/access.log | \
  awk '{print $1}' | sort | uniq -c | sort -rn | head -5

# Step 4: See context — what happened before each 500?
grep -B 5 "\" 500 " /var/log/app.log | head -50
```

🔷 Common interview questions

Q: What's the difference between `grep -E` and `grep -P`?

> `-E` uses POSIX Extended Regular Expressions (ERE) — portable across all Unix systems. `-P` uses Perl-Compatible Regular Expressions (PCRE) which supports lookaheads, lookbehinds, `\d`, `\w`, `\s` etc. PCRE is more powerful but not available on all systems (notably, macOS `grep` may not support `-P`).

Q: How do you search for a literal string with special characters like `.*[]`?

```
grep -F "192.168.1.1"      # -F = fixed string, no regex interpretation
grep "192\.168\.1\.1"      # Escape the dots manually
```

Q: How do you grep across gzipped log files?

```
zgrep "ERROR" /var/log/app.log.gz
# Or:
zcat /var/log/app.log.gz | grep "ERROR"
```

⚠ Tricky gotchas

```
# GOTCHA 1: grep returns exit code 1 if no match found
# This breaks scripts with set -e!
grep "pattern" file || true    # Safe - don't exit on no match

# GOTCHA 2: grep -r follows symlinks (can cause infinite loops)
grep -r --no-dereference "pattern" /

# GOTCHA 3: Binary files
grep "text" binary_file     # Prints "Binary file matches"
grep -a "text" binary_file  # -a treats binary as text

# GOTCHA 4: Grepping for a pattern starting with -
grep -- "-v" file.txt       # Use -- to signal end of options

# GOTCHA 5: Performance - grep before pipe, not after
# SLOW:
cat huge_file.log | head -1000 | grep "ERROR"
# FAST:
grep "ERROR" huge_file.log | head -1000
```

🔷 Connections to other concepts

- Feeds into `awk`, `sed`, `cut` pipelines
- Used with `xargs` for batch operations
- Exit codes matter with `set -e` in scripts
- `zgrep`, `bzgrep` for compressed log rotation workflows

4.2 `cut`, `sort`, `uniq`, `wc` — Pipeline Building Blocks

🔷 What they are

These four tools are the **lego bricks** of text pipelines. Rarely used alone — almost always chained together.

```
Input → grep (filter) → cut (extract fields) → sort (order) → uniq
(deduplicate) → wc (count)
```

🔷 `cut` — Extract columns/fields

```
# Sample: /etc/passwd format
# username:password:UID:GID:comment:home:shell

# Extract just usernames (field 1, delimiter :)
cut -d: -f1 /etc/passwd
```

```
# Extract username and shell (fields 1 and 7)
cut -d: -f1,7 /etc/passwd

# Extract by character position (columns 1-10)
cut -c1-10 /var/log/syslog

# Extract from field 3 onwards
cut -d: -f3- /etc/passwd

# Real example: extract IPs from nginx log
cut -d' ' -f1 /var/log/nginx/access.log   # Extracts IP column
```

> ⚠ **`cut` weakness**: It can only split on a single character delimiter. It cannot handle multiple spaces or variable whitespace. Use `awk` for that.

```
# This FAILS with variable whitespace:
echo "  hello   world" | cut -d' ' -f2   # Returns empty!
# Use awk instead:
echo "  hello   world" | awk '{print $1}'  # Returns "hello"
```

🔷 `sort` — Sort lines

```
# Alphabetical (default)
sort /etc/passwd

# Reverse order
sort -r file.txt

# Numeric sort (critical difference!)
sort file.txt       # Alphabetic: 1, 10, 2, 20, 3
sort -n file.txt    # Numeric:    1, 2, 3, 10, 20

# Sort by specific field (field 3, delimiter :)
sort -t: -k3 -n /etc/passwd     # Sort by UID numerically

# Sort by multiple keys
sort -t: -k3 -k1 /etc/passwd    # Sort by UID, then username

# Sort by file size (human readable)
du -sh * | sort -h              # -h understands K, M, G

# Remove duplicates while sorting
sort -u file.txt                # Sort + unique in one pass

# Sort in memory (for huge files, set buffer)
sort -S 2G huge_file.txt
```

🔷 `uniq` — Deduplicate adjacent lines

> ⚠ **Critical**: `uniq` only removes **adjacent** duplicates. **Always `sort` first.**

```
# Wrong — won't work without sort:
echo -e "apple\nbanana\napple" | uniq
# Output: apple, banana, apple  ← NOT deduplicated!

# Correct:
echo -e "apple\nbanana\napple" | sort | uniq
# Output: apple, banana

# Count occurrences (most used in interviews!)
sort file.txt | uniq -c
# Output:
#   3 apple
#   1 banana
#   2 cherry

# Sort by frequency (most common first)
sort file.txt | uniq -c | sort -rn

# Show only duplicates
sort file.txt | uniq -d

# Show only unique (non-duplicated) lines
sort file.txt | uniq -u
```

🔷 `wc` — Word/Line/Character Count

```
wc file.txt          # lines  words  chars  filename
wc -l file.txt       # lines only (most common)
wc -w file.txt       # words only
wc -c file.txt       # bytes
wc -m file.txt       # characters (multibyte-aware)

# Count files in directory
ls /etc | wc -l

# Count running processes
ps aux | wc -l

# Count ERROR lines in log
grep "ERROR" app.log | wc -l
```

🔷 The Power Pipeline Pattern

```
# "Top 10 most frequent IPs hitting your server"
cut -d' ' -f1 /var/log/nginx/access.log \
  | sort \
  | uniq -c \
  | sort -rn \
  | head -10

# "How many unique users logged in today?"
grep "Accepted password" /var/log/auth.log \
  | awk '{print $9}' \
  | sort -u \
  | wc -l

# "Top 5 most common HTTP status codes"
awk '{print $9}' /var/log/nginx/access.log \
  | sort \
  | uniq -c \
  | sort -rn \
  | head -5
```

🔷 Short crisp interview answer

> "`cut` extracts specific columns, `sort` orders lines, `uniq` deduplicates adjacent lines (so always sort first), and `wc` counts lines/words/chars. These four are the foundation of log analysis pipelines — I chain them constantly: `cut` to extract a field, `sort` to group it, `uniq -c` to count occurrences, `sort -rn` to rank by frequency."

4.3 `tr` — Character Translation

🔷 What it is

`tr` (**tr**anslate) operates character-by-character on stdin and either **replaces**, **deletes**, or **squeezes** characters. It never reads files directly — always via stdin.

```
# Basic syntax
tr [options] SET1 [SET2]
```

🔷 Key uses with examples

```
# Uppercase to lowercase
echo "HELLO WORLD" | tr 'A-Z' 'a-z'
# hello world

# Lowercase to uppercase
```

```
echo "hello" | tr 'a-z' 'A-Z'
# HELLO

# Replace character (colons to spaces)
echo "root:x:0:0" | tr ':' ' '
# root x 0 0

# Delete characters (-d)
echo "h3ll0 w0rld" | tr -d '0-9'
# hll wrld

echo "hello world" | tr -d ' '
# helloworld

# Squeeze repeated characters (-s)
echo "heeelllloooo   world" | tr -s 'a-z'
# helo world

echo "too    many    spaces" | tr -s ' '
# too many spaces

# Delete newlines (join lines)
cat file.txt | tr -d '\n'

# Convert newlines to spaces
cat file.txt | tr '\n' ' '

# ROT13 encoding
echo "Hello" | tr 'A-Za-z' 'N-ZA-Mn-za-m'
# Uryyb

# Remove non-printable characters
cat binary_or_weird_file | tr -cd '[:print:]\n'

# Character classes
tr '[:upper:]' '[:lower:]'   # Portable uppercase→lowercase
tr -d '[:space:]'            # Delete all whitespace
tr -d '[:punct:]'            # Delete punctuation
```

🔷 Real production use

```
# Clean up a CSV with Windows line endings (\r\n)
tr -d '\r' < windows_file.csv > unix_file.csv

# Normalize a config file — squeeze multiple spaces
cat messy.conf | tr -s ' ' > clean.conf

# Extract only digits from output
echo "Server uptime: 3652 days" | tr -cd '0-9'
# 3652
```

```
# Generate a random password (using /dev/urandom)
cat /dev/urandom | tr -dc 'A-Za-z0-9!@#$%' | head -c 20
```

⚠ Gotcha

```
# tr does NOT support regex or multi-char strings
tr 'hello' 'world'   # This maps h→w, e→o, l→r, l→l, o→d
                     # NOT the string "hello" → "world"
# For string replacement, use sed:
sed 's/hello/world/g'
```

4.4 sed — Stream Editor 🟡

🔷 What it is

sed reads input line-by-line, applies editing commands, and outputs the result. It's a **non-interactive editor** — perfect for scripted find-and-replace, deletion, and transformation of text files.

```
stdin/file
     │
     ▼
┌──────────────────────┐
│  Read line into      │
│  pattern space       │
└──────────────────────┘
         │
         ▼
┌──────────────────────┐
│  Apply sed commands  │  ◄── address + command
└──────────────────────┘
         │
         ▼
┌──────────────────────┐
│  Print pattern space │ ──► stdout
│  (unless -n)         │
└──────────────────────┘
         │
    Next line
```

🔷 Core syntax

```
sed '[address]command[/flags]' file
```

- **address** — which line(s) to operate on (optional)
- **command** — what to do (`s`, `d`, `p`, `i`, `a`, `q`, etc.)
- **flags** — `g` (global), `i` (case-insensitive), `p` (print)

🔷 Substitution — `s` command (most used)

```
# Basic: replace first occurrence per line
sed 's/old/new/' file.txt

# Global: replace ALL occurrences per line
sed 's/old/new/g' file.txt

# Case-insensitive replace
sed 's/error/ERROR/gi' file.txt

# In-place edit (MODIFIES the file!)
sed -i 's/old/new/g' file.txt

# In-place with backup (ALWAYS do this in production!)
sed -i.bak 's/old/new/g' file.txt
# Creates file.txt.bak before modifying

# Replace only on specific line number
sed '3s/old/new/' file.txt       # Only line 3

# Replace in line range
sed '2,5s/old/new/g' file.txt    # Lines 2-5

# Replace from line to end
sed '10,$s/old/new/g' file.txt   # Line 10 to EOF

# Replace on lines matching a pattern
sed '/ERROR/s/disk/DISK/g' file.txt  # Only on lines containing ERROR
```

🔷 Deletion — `d` command

```
# Delete specific line
sed '5d' file.txt

# Delete range of lines
sed '2,5d' file.txt

# Delete lines matching pattern
sed '/^#/d' file.txt                    # Delete comment lines
sed '/^$/d' file.txt                    # Delete blank lines
sed '/^[[:space:]]*$/d' file.txt        # Delete blank/whitespace-only
```

```
lines

# Delete from pattern to end of file
sed '/ERROR/,$d' file.txt
```

🔷 Print — p command with -n

```
# -n suppresses default printing, p explicitly prints
sed -n '5p' file.txt                    # Print only line 5
sed -n '2,8p' file.txt                  # Print lines 2-8
sed -n '/ERROR/p' file.txt              # Print only matching lines (like
grep)
sed -n '/START/,/END/p' file.txt        # Print between two patterns
```

🔷 Insert and Append

```
# Insert line BEFORE line 3
sed '3i\This is inserted before line 3' file.txt

# Append line AFTER line 3
sed '3a\This is appended after line 3' file.txt

# Append after matching line
sed '/ERROR/a\ALERT: Check the system!' file.txt
```

🔷 Multiple commands

```
# Use -e for multiple expressions
sed -e 's/foo/bar/g' -e 's/baz/qux/g' file.txt

# Or use semicolons
sed 's/foo/bar/g; s/baz/qux/g' file.txt

# Or a script file
sed -f script.sed file.txt
```

🔷 Advanced: capture groups & back-references

```
# Swap two words using capture groups
echo "John Smith" | sed 's/\(.*\) \(.*\)/\2 \1/'
```

```
# Smith John

# With ERE (-E), cleaner syntax:
echo "John Smith" | sed -E 's/(.+) (.+)/\2 \1/'
# Smith John

# Extract date from log and reformat
echo "2024-01-15 ERROR" | sed -E 's/([0-9]{4})-([0-9]{2})-([0-9]
{2})/\3\/\2\/\1/'
# 15/01/2024 ERROR
```

🔷 Real-world production examples

```
# 1. Update config file in-place (deployment script)
sed -i.bak "s/DB_HOST=localhost/DB_HOST=${NEW_DB_HOST}/g"
/etc/app/config.env

# 2. Remove all comment lines and blank lines from config
sed '/^#/d; /^$/d' /etc/nginx/nginx.conf

# 3. Add a line after a specific config section
sed '/\[database\]/a host = newserver.internal' config.ini

# 4. Extract lines between START and END markers
sed -n '/^---BEGIN CERT---/,/^---END CERT---/p' file.pem

# 5. Fix Windows line endings across all files
find /opt/app -name "*.sh" -exec sed -i 's/\r//' {} \;

# 6. Comment out a specific line in a config
sed -i '/^ServerName/s/^/#/' /etc/apache2/httpd.conf
```

🔷 Short crisp interview answer

> "`sed` is a stream editor that processes text line-by-line. I use it most for in-place substitution with `sed -i 's/old/new/g'` in deployment scripts, deleting comment/blank lines from configs, and extracting content between patterns. The key internals are the pattern space (current line buffer) and the address system that lets you target specific lines or ranges."

⚠ Gotchas

```
# GOTCHA 1: -i behavior differs between GNU sed and macOS sed
sed -i 's/old/new/g' file        # GNU/Linux — works
sed -i '' 's/old/new/g' file     # macOS — requires empty string argument!
```

```
# GOTCHA 2: sed modifies pattern space, not the file, unless -i
sed 's/old/new/g' file.txt       # Prints to stdout, FILE IS UNCHANGED

# GOTCHA 3: Delimiter doesn't have to be /
# Useful when your pattern contains slashes (like file paths!)
sed 's|/old/path|/new/path|g' file.txt   # Use | as delimiter
sed 's#/old/path#/new/path#g' file.txt   # Or #

# GOTCHA 4: & in replacement means "the whole match"
echo "hello" | sed 's/hello/[&]/'
# [hello]
```

4.5 awk — Field Processor & Mini Language 🟡🔥

🔷 What it is

awk is a **complete programming language** disguised as a command-line tool. It processes text record-by-record (usually line-by-line), splitting each record into **fields**, and lets you apply conditions, perform math, and format output.

Named after its creators: **A**ho, **W**einberger, **K**ernighan.

🔷 Internal model

```
Input file/stream
       │
       ▼
┌──────────────────────────┐
│  BEGIN block (runs once) │ ◄── Initialize variables, print headers
└──────────────────────────┘
             │
             ▼
    ┌──────────────────┐
    │  Read next record  │ ◄── Default: one line = one record (RS="\n")
    └──────────────────┘
             │
             ▼
    ┌──────────────────┐
    │  Split into fields  │ ◄── Default: whitespace delimiter (FS=" ")
    │  $1 $2 $3 ... $NF  │     $0 = entire line, NF = number of fields
    └──────────────────┘
             │
             ▼
    ┌──────────────────┐
    │  Apply rules:      │
    │  pattern { action } │ ◄── If pattern matches, run action
    └──────────────────┘
             │
         More records?
             │
             ▼
    ┌──────────────────┐
```

```
      │  END block (once)    │ ←── Print totals, cleanup
      └──────────────────────┘
```

🔷 Built-in variables

Variable	Meaning
`$0`	Entire current line
`$1`, `$2`...	Field 1, field 2...
`$NF`	Last field
`NR`	Current record (line) number
`NF`	Number of fields in current record
`FS`	Field separator (default: whitespace)
`RS`	Record separator (default: newline)
`OFS`	Output field separator
`ORS`	Output record separator
`FILENAME`	Current input file name

🔷 Core examples — from simple to powerful

```
# Sample log:
# 192.168.1.1 - frank [15/Jan/2024] "GET /index.html" 200 1234
# 10.0.0.1   - bob   [15/Jan/2024] "POST /api/login" 401 567
# 192.168.1.1 - frank [15/Jan/2024] "GET /dashboard" 200 9876

# Print specific field
awk '{print $1}' access.log          # Print IPs (field 1)
awk '{print $7}' access.log          # Print HTTP status codes
awk '{print $1, $7}' access.log      # Print IP and status (OFS=" ")
awk '{print $1 "\t" $7}' access.log  # Tab-separated

# Print last field
awk '{print $NF}' access.log         # Response size

# Print second-to-last field
awk '{print $(NF-1)}' access.log     # HTTP status

# Change delimiter
awk -F: '{print $1}' /etc/passwd             # -F sets field separator
awk -F'[,:]' '{print $1}' file.txt           # Multiple delimiters (regex)

# Pattern matching — print only matching lines
awk '/ERROR/ {print}' app.log                # Lines with ERROR
```

```
awk '$7 == "200" {print $1}' access.log      # IPs with 200 status
awk '$7 >= 500 {print}' access.log           # All 5xx errors
awk 'NR >= 10 && NR <= 20 {print}' file      # Lines 10-20

# Arithmetic
awk '{sum += $NF} END {print "Total bytes:", sum}' access.log
awk '{sum += $NF} END {print "Avg bytes:", sum/NR}' access.log

# Count occurrences
awk '{count[$7]++} END {for (code in count) print count[code], code}'
access.log

# BEGIN and END blocks
awk 'BEGIN {print "=== Report ==="}
     /ERROR/ {errors++}
     END {print "Total errors:", errors}' app.log

# Conditional logic
awk '{
  if ($7 >= 500) print "ERROR:", $0
  else if ($7 >= 400) print "WARN:", $0
  else print "OK:", $0
}' access.log

# String operations
awk '{gsub(/ERROR/, "CRITICAL"); print}' app.log   # Global substitute
awk '{print toupper($0)}' file.txt                  # Uppercase entire
line
awk '{print length($0)}' file.txt                   # Line length

# Reformatting output
awk -F: '{printf "User: %-15s Shell: %s\n", $1, $7}' /etc/passwd
```

🔷 Short crisp interview answer

> "`awk` is a text-processing language that splits each line into fields and lets you filter, compute, and reformat. I use it for extracting specific columns from logs, summing values, counting occurrences with associative arrays, and generating reports. Its key power over `cut` is that it handles variable whitespace and can do math and conditionals in the same pass."

🔷 Deep version

> "Internally, `awk` reads records (lines by default, controlled by `RS`), splits them into fields by `FS`, then evaluates each `pattern { action }` rule against the record. The entire program runs in a single pass through the file — BEGIN runs once before any input, END runs once after all input. `awk` maintains an associative array (hash map) natively which makes frequency counting, grouping, and aggregation very efficient. For production log analysis, `awk` is often faster than a Python script for simple aggregation tasks because it avoids interpreter startup overhead and processes data in a streaming fashion."

🔷 Real production example

```
# Generate an incident report from nginx logs:
awk '
BEGIN {
  print "=== Incident Report ===="
  print "Time\t\tStatus\tIP\t\tEndpoint"
  print "----\t\t------\t--\t\t--------"
}
$9 >= 500 {
  gsub(/[\[\]]/, "", $4)
  printf "%s\t%s\t%-15s\t%s\n", $4, $9, $1, $7
  errors++
  ip_count[$1]++
}
END {
  print "\n=== Summary ===="
  print "Total 5xx errors:", errors
  print "\nTop offending IPs:"
  for (ip in ip_count)
    print ip_count[ip], ip | "sort -rn | head -5"
}
' /var/log/nginx/access.log
```

⚠ Gotchas

```
# GOTCHA 1: awk field separator -F vs FS
awk -F: '{print $1}' file           # Command line
awk 'BEGIN{FS=":"} {print $1}' file # Equivalent, in BEGIN block

# GOTCHA 2: String vs numeric comparison
awk '$3 == "10"'   # String comparison
awk '$3 == 10'     # Numeric comparison — different behavior!

# GOTCHA 3: print vs printf
awk '{print $1, $2}'               # Adds OFS between, ORS at end
awk '{printf "%s %s\n", $1, $2}'   # Explicit control

# GOTCHA 4: Modifying $0
awk '{$2="REDACTED"; print}' file  # Rebuilds $0 with OFS
# If OFS is default (space), columns may shift
```

4.6 `grep` vs `sed` vs `awk` — When to Use Which ⚠🔥

🔷 The Mental Model

```
                    ┌──────────────────────────────────────┐
                    │        WHAT DO YOU NEED TO DO?       │
                    └──────────────────────────────────────┘
                                      │
                                      │
              ┌───────────────────────┼───────────────────────┐
              │                       │                       │
              ▼                       ▼                       ▼
     Does it MATCH/NOT          TRANSFORM              COMPUTE/
     MATCH a pattern?           text in place?         AGGREGATE?
              │                       │                       │
              ▼                       ▼                       ▼
           GREP                     SED                    AWK
     (filter lines)          (edit text stream)      (process & calculate)
```

🔷 Side-by-side comparison

Capability	grep	sed	awk
Filter lines by pattern	✅ Best	✅ Yes (`-n /pat/p`)	✅ Yes
Find & replace text	❌ No	✅ Best	✅ Yes (`gsub`)
In-place file edit	❌ No	✅ Yes (`-i`)	❌ Not natively
Extract specific fields	❌ No	🟡 Limited	✅ Best
Arithmetic/math	❌ No	❌ No	✅ Yes
Conditionals	❌ No	🟡 Limited	✅ Full
Associative arrays	❌ No	❌ No	✅ Yes
Multiple passes	❌ No	🟡 With labels	✅ Yes
Speed (simple filter)	✅ Fastest	🟡 Fast	🟡 Fast
Learning curve	Low	Medium	Higher

🔷 Decision rules

```
# USE grep WHEN:
# → You want to FILTER lines (show/hide lines matching a pattern)
grep "ERROR" app.log
grep -v "DEBUG" app.log

# USE sed WHEN:
# → You want to SUBSTITUTE, INSERT, or DELETE text
# → You want to edit a file IN-PLACE
sed -i 's/localhost/prod-db/g' config.env
sed '/^#/d' config.conf          # Strip comments
```

```
# USE awk WHEN:
# → You want to work with COLUMNS/FIELDS
# → You need ARITHMETIC or AGGREGATION
# → You want CONDITIONAL logic beyond simple line filtering
awk '{sum += $5} END {print sum}' metrics.log
awk -F: '$3 > 1000 {print $1}' /etc/passwd   # Non-system users
```

🔷 Same task, three tools — comparison

Task: Find all lines with "ERROR" and print only the timestamp (field 1)

```
# grep alone can't extract fields — needs a pipe
grep "ERROR" app.log | awk '{print $1}'

# sed can print first word but it's awkward
grep "ERROR" app.log | sed 's/ .*//'

# awk does it all in one  ← BEST for this task
awk '/ERROR/ {print $1}' app.log
```

Task: Replace "staging" with "production" in a config file

```
# grep — can't replace
# awk — can, but not cleanly in-place
awk '{gsub(/staging/, "production"); print}' config > tmp && mv tmp config

# sed — BEST for this task
sed -i 's/staging/production/g' config
```

Task: Count how many times each IP appears in a log

```
# grep + sort + uniq — works for simple case
grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' access.log | sort | uniq -c |
sort -rn

# awk — BEST because you can add filters easily
awk '{count[$1]++} END {for(ip in count) print count[ip], ip}' access.log
| sort -rn
```

🔷 Short crisp interview answer

> "The rule of thumb I follow: use `grep` to filter lines, `sed` to transform or replace text (especially in-place file edits), and `awk` when I need field extraction, arithmetic, or aggregation. In practice they

> chain together — `grep` first to reduce data volume, then `awk` to process the filtered output. `sed` is my go-to for deployment scripts that need to update config files."

4.7 Advanced `awk` — Multi-rule, Associative Arrays, Formatted Output 🔴

🔷 Associative arrays (hash maps in awk)

```
# Frequency count — the most powerful awk pattern
awk '{count[$1]++} END {
  for (ip in count)
    printf "%-20s %d\n", ip, count[ip]
}' access.log | sort -k2 -rn

# Two-dimensional association
awk '{status_per_ip[$1][$9]++} END {
  for (ip in status_per_ip)
    for (code in status_per_ip[ip])
      print ip, code, status_per_ip[ip][code]
}' access.log

# Sum bytes per IP
awk '{bytes[$1] += $NF} END {
  for (ip in bytes)
    printf "%-20s %.2f KB\n", ip, bytes[ip]/1024
}' access.log | sort -k2 -rn
```

🔷 Multi-rule awk programs

```
awk '
/^#/ { next }                    # Skip comments
/ERROR/ { errors++; print "ERROR:", $0 }
/WARNING/ { warnings++ }
/INFO/ { info++ }
END {
  print "\n=== Log Summary ===="
  printf "Errors:   %d\n", errors
  printf "Warnings: %d\n", warnings
  printf "Info:     %d\n", info
  printf "Error rate: %.1f%%\n", (errors/(errors+warnings+info))*100
}
' application.log
```

🔷 Multi-file processing

```
# FNR = record number within current file
# NR  = total record number across all files
awk '
FNR == 1 { print "--- Processing:", FILENAME }
/ERROR/ { print NR":", $0 }
' /var/log/app1.log /var/log/app2.log
```

🔷 Formatted reporting

```
awk -F: '
BEGIN {
  printf "%-20s %-6s %-30s\n", "Username", "UID", "Home Directory"
  printf "%-20s %-6s %-30s\n", "--------", "---", "--------------"
}
$3 >= 1000 {   # Non-system users
  printf "%-20s %-6s %-30s\n", $1, $3, $6
}
END {
  print "\nTotal regular users:", NR
}
' /etc/passwd
```

🔷 `getline` — Read extra input

```
awk '/ERROR/ {
  print "Error found:", $0
  getline nextline    # Read next line into variable
  print "Next line:", nextline
}' app.log
```

4.8 Log Parsing Pipelines — Real Incident Analysis 🔴

🔷 The 2 AM Incident Toolkit

Scenario 1: High 5xx error rate

```
# 1. Current error rate (last 1000 lines)
tail -1000 /var/log/nginx/access.log | \
  awk '{status[$9]++} END {
    total = 0
    for (s in status) total += status[s]
    for (s in status)
```

```
        printf "%s: %d (%.1f%%)\n", s, status[s], (status[s]/total)*100
    }' | sort

  # 2. Error spike timeline (errors per minute)
  grep '" 5' /var/log/nginx/access.log | \
    awk '{print $4}' | \
    cut -c2-18 | \
    cut -d: -f1-3 | \
    sort | uniq -c

  # 3. Which endpoints are 500ing?
  awk '$9 >= 500 {print $7}' /var/log/nginx/access.log | \
    sort | uniq -c | sort -rn | head -10

  # 4. Is it one bad actor IP?
  awk '$9 >= 500 {print $1}' /var/log/nginx/access.log | \
    sort | uniq -c | sort -rn | head -5
```

Scenario 2: Disk space alert

```
  # Find what's eating disk
  du -sh /* 2>/dev/null | sort -rh | head -20

  # Find files larger than 1GB
  find / -xdev -size +1G -exec ls -lh {} \; 2>/dev/null

  # Find largest log files specifically
  find /var/log -name "*.log" -exec du -sh {} \; | sort -rh | head -10

  # Check for deleted-but-open files eating space
  lsof +L1 | awk '$7 > 1073741824 {print $1, $2, $7/1024/1024 "MB", $9}'
```

Scenario 3: Parse structured logs (JSON)

```
  # App logs in JSON:
  {"level":"ERROR","msg":"timeout","service":"api","duration_ms":3200}

  # Extract all ERROR messages
  grep '"level":"ERROR"' app.log | jq -r '.msg'

  # Average duration of slow requests
  grep '"level":"ERROR"' app.log | \
    jq -r '.duration_ms' | \
    awk '{sum+=$1; count++} END {print "Avg:", sum/count, "ms"}'

  # Group errors by service
  grep '"level":"ERROR"' app.log | \
```

```
  jq -r '.service' | \
  sort | uniq -c | sort -rn
```

Scenario 4: Auth log analysis

```
# Detect brute force — IPs with many failed logins
grep "Failed password" /var/log/auth.log | \
  awk '{print $(NF-3)}' | \
  sort | uniq -c | sort -rn | head -10

# Successful logins (after how many failures?)
grep -E "Failed|Accepted" /var/log/auth.log | \
  awk '{
    if (/Failed/) fails[$11]++
    if (/Accepted/) print $11, "logged in after", fails[$11], "failures"
  }'
```

4.9 `jq` — JSON Parsing for Cloud APIs 🔴

🔷 What it is

`jq` is a lightweight, command-line JSON processor. In the cloud/DevOps world, almost every API (AWS CLI, Kubernetes, GitHub, etc.) returns JSON. `jq` is how you extract and transform that data in shell scripts.

🔷 Core syntax

```
jq [options] 'filter' [file]
```

```
# Sample JSON:
cat > /tmp/servers.json << 'EOF'
{
  "servers": [
    {"name": "web-01", "status": "running", "ip": "10.0.0.1", "cpu": 45},
    {"name": "web-02", "status": "stopped", "ip": "10.0.0.2", "cpu": 0},
    {"name": "db-01",  "status": "running", "ip": "10.0.0.3", "cpu": 87}
  ],
  "total": 3
}
EOF

# Pretty print (just validate/format)
jq '.' /tmp/servers.json

# Get top-level field
```

```
jq '.total' /tmp/servers.json              # 3

# Get nested field
jq '.servers[0].name' /tmp/servers.json    # "web-01"

# Get array element field
jq '.servers[].name' /tmp/servers.json
# "web-01"
# "web-02"
# "db-01"

# Raw output (no quotes) — for use in scripts!
jq -r '.servers[].name' /tmp/servers.json

# Filter array by condition
jq '.servers[] | select(.status == "running")' /tmp/servers.json

# Extract specific fields from filtered results
jq -r '.servers[] | select(.status == "running") | .name + " " + .ip'
/tmp/servers.json
# web-01 10.0.0.1
# db-01 10.0.0.3

# Transform to new JSON structure
jq '[.servers[] | {host: .name, addr: .ip}]' /tmp/servers.json

# Arithmetic and conditions
jq '.servers[] | select(.cpu > 80) | .name' /tmp/servers.json
# "db-01"

# Length of array
jq '.servers | length' /tmp/servers.json   # 3

# Map — transform every element
jq '.servers | map(.name)' /tmp/servers.json  # ["web-01","web-02","db-
01"]

# Sort array
jq '.servers | sort_by(.cpu) | reverse | .[].name' /tmp/servers.json
```

🔷 Real-world: AWS CLI + jq

```
# List all running EC2 instance IDs and their IPs
aws ec2 describe-instances \
  --filters "Name=instance-state-name,Values=running" \
  | jq -r '.Reservations[].Instances[] | .InstanceId + " " +
.PrivateIpAddress'

# Get all pod names in a namespace from kubectl
kubectl get pods -n production -o json | \
```

```
  jq -r '.items[] | select(.status.phase != "Running") | .metadata.name'

# Extract a secret value from AWS Secrets Manager
aws secretsmanager get-secret-value --secret-id myapp/db \
  | jq -r '.SecretString | fromjson | .password'

# Count resources by type across a Terraform state file
cat terraform.tfstate | jq '[.resources[].type] | group_by(.) | map({type:
.[0], count: length})'
```

🔷 Short crisp interview answer

> "`jq` is the essential tool for working with JSON in shell scripts. Every cloud CLI returns JSON — AWS, GCP, kubectl. I use it to extract fields with `.field`, filter arrays with `select()`, transform structures with `map()`, and always use `-r` for raw string output when piping into other commands. Without `jq`, working with cloud APIs in bash would require Python for every little thing."

4.10 `perl` One-liners — When awk/sed Aren't Enough 🔴

🔷 What it is and why

Perl was the original "Swiss Army chainsaw" for text processing. While `awk` and `sed` cover 90% of cases, Perl one-liners shine when you need complex regex (lookaheads, named groups), file encoding handling, multi-line pattern matching, or cross-platform consistency.

```
perl -e 'code'              # Run inline code
perl -n 'code'              # Loop over lines (like awk), no auto-print
perl -p 'code'              # Loop over lines WITH auto-print (like sed)
perl -i 'code' file         # In-place edit (like sed -i)
perl -lane 'code'           # -l (auto-chomp), -a (auto-split to @F), -n,
-e
```

🔷 Key examples

```
# Basic substitution (like sed)
perl -pe 's/old/new/g' file.txt

# In-place edit (works identically on Linux AND macOS!)
perl -i -pe 's/old/new/g' file.txt

# With backup
perl -i.bak -pe 's/old/new/g' file.txt

# Print only matching lines (like grep)
perl -ne 'print if /ERROR/' file.txt
```

```
# Auto-split into fields (like awk) with -a
# @F = array of fields, $F[0] = first field
perl -lane 'print $F[0]' file.txt          # Print first field
perl -lane 'print $F[-1]' file.txt         # Print last field (negative
index!)

# Perl regex features awk/sed lack:

# Named capture groups
perl -ne 'if (/(?<ip>\d+\.\d+\.\d+\.\d+)/) { print "$+{ip}\n" }'
access.log

# Lookbehind
perl -pe 's/(?<=ERROR: )\w+/REDACTED/g' app.log

# Non-greedy matching (strip HTML tags)
perl -pe 's/<.*?>//g' file.html

# Math
perl -lane '$sum += $F[2]; END { print "Sum:", $sum }' file.txt

# Multi-line matching (awk/sed struggle with this)
perl -0777 -pe 's/START.*?END/REPLACED/gs' file.txt
# -0777 slurps entire file, /s makes . match newlines

# Print lines between two patterns (inclusive)
perl -ne 'print if /START/../END/' file.txt

# Delete blank lines
perl -ne 'print unless /^$/' file.txt

# URL decode
perl -pe 's/%([0-9A-Fa-f]{2})/chr(hex($1))/ge' encoded.txt

# Base64 decode
perl -MMIME::Base64 -ne 'print decode_base64($_)' encoded.txt
```

🔷 `perl` vs `awk` vs `sed` — When to reach for Perl

Use case	Tool
Simple line filter	`grep`
Simple substitution	`sed`
Field extraction + math	`awk`
Complex regex (lookahead, lookbehind)	`perl`
Multi-line pattern matching	`perl`

Use case	Tool
Cross-platform consistency (Linux + macOS)	`perl`
URL/Base64/encoding work	`perl`
Need modules (JSON, CSV, etc.)	`perl` or `python`

🔷 Real production use

```
# 1. Strip ANSI color codes from log files
perl -pe 's/\e\[[0-9;]*m//g' colored.log > clean.log

# 2. Extract all email addresses from a file
perl -nle 'print for /[\w.+-]+@[\w-]+\.[\w.]+/g' file.txt

# 3. In-place config update that works on BOTH Linux and macOS
perl -i -pe "s/HOST=.*/HOST=${NEW_HOST}/" config.env

# 4. Parse a multi-line stack trace
perl -0777 -ne '
  while (/Exception: (.*?)(?=\n\n|\z)/sg) {
    print "EXCEPTION: $1\n\n"
  }
' app.log
```

🏆 Category 4 — Complete Mental Model

```
TEXT PROCESSING DECISION TREE
═══════════════════════════════

Input text
    │
    ├── Need to FIND/FILTER lines?
    │       └── grep (fastest, simplest)
    │
    ├── Need to REPLACE/EDIT text?
    │       ├── Simple substitution → sed -i 's/old/new/g'
    │       ├── Complex regex (lookahead) → perl -i -pe
    │       └── Delete lines/chars → sed 'd' or tr -d
    │
    ├── Need to work with COLUMNS/FIELDS?
    │       ├── Single delimiter, simple → cut -d: -f1
    │       ├── Variable whitespace, math → awk
    │       └── Complex conditions → awk
    │
    ├── Need to AGGREGATE/COUNT/SUM?
    │       ├── Simple counting → sort | uniq -c
```

```
    │       └── Complex grouping → awk associative arrays
    │
    ├── Input is JSON?
    │       └── jq (always, no exceptions)
    │
    └── Multi-line patterns? Complex regex? Cross-platform?
            └── perl one-liners
```

⚠ Master Gotcha List

#	Gotcha	Fix
1	`uniq` requires sorted input	Always `sort | uniq`, never just `uniq`
2	`cut` can't handle variable whitespace	Use `awk $N` instead
3	`sed -i` differs: GNU vs macOS	GNU: `sed -i`; macOS: `sed -i ''`
4	`grep` returns exit code 1 on no match	Use `grep "pat" file || true` in scripts
5	`awk` string vs numeric comparison	`$3 == "10"` vs `$3 == 10` are different
6	`sed` delimiter clashes with `/`	Use `|` or `#` as alternate delimiter
7	`jq` returns quoted strings by default	Use `jq -r` for raw string output
8	`tr` works on chars, not strings	Use `sed` for string replacement
9	`awk $NF` is last field	`$(NF-1)` is second-to-last
10	`sort -n` vs `sort -h`	`-n` for numbers, `-h` for human sizes (K/M/G)

🔥 Top Interview Questions

Q1: Write a one-liner to find the top 5 IPs in an nginx log.

```
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn |
head -5
```

Q2: Replace a string in 50 files simultaneously.

```
find /etc/app -name "*.conf" | xargs sed -i 's/staging/production/g'
# Cross-platform safe:
find /etc/app -name "*.conf" | xargs perl -i -pe 's/staging/production/g'
```

Q3: Sum the 4th column of a CSV file.

```
awk -F, '{sum += $4} END {print sum}' data.csv
```

Q4: Print lines 100-200 of a file without loading the whole file.

```
sed -n '100,200p' hugefile.log
# Or:
awk 'NR>=100 && NR<=200' hugefile.log
```

Q5: Extract all unique HTTP status codes from an access log.

```
awk '{print $9}' access.log | sort -u
```

Q6: Count occurrences of each log level in a JSON log.

```
jq -r '.level' app.log | sort | uniq -c | sort -rn
```

Q7: Delete all blank lines from a file in-place.

```
sed -i '/^$/d' file.txt
# Including whitespace-only lines:
sed -i '/^[[:space:]]*$/d' file.txt
```

Q8: What is the difference between `NR` and `FNR` in awk?

> `NR` is the total record number across all input files. `FNR` resets to 1 at the start of each new file. Use `FNR == 1` to detect the first line of each file when processing multiple files.

Q9: How do you safely do in-place sed edits in production?

```
# Always make a backup first!
sed -i.bak 's/old/new/g' config.env
# Verify the change:
diff config.env config.env.bak
# Only remove backup when confident:
rm config.env.bak
```

Q10: How do you handle a log file that's being written to in real-time?

```
tail -f /var/log/app.log | grep --line-buffered "ERROR" | \
  awk '{print strftime("%Y-%m-%d %H:%M:%S"), $0; fflush()}'
```

```
# --line-buffered on grep and fflush() in awk prevent buffering issues
```

Document covers: grep, cut, sort, uniq, wc, tr, sed, awk, jq, perl — complete with examples, internals, production patterns, and interview answers.